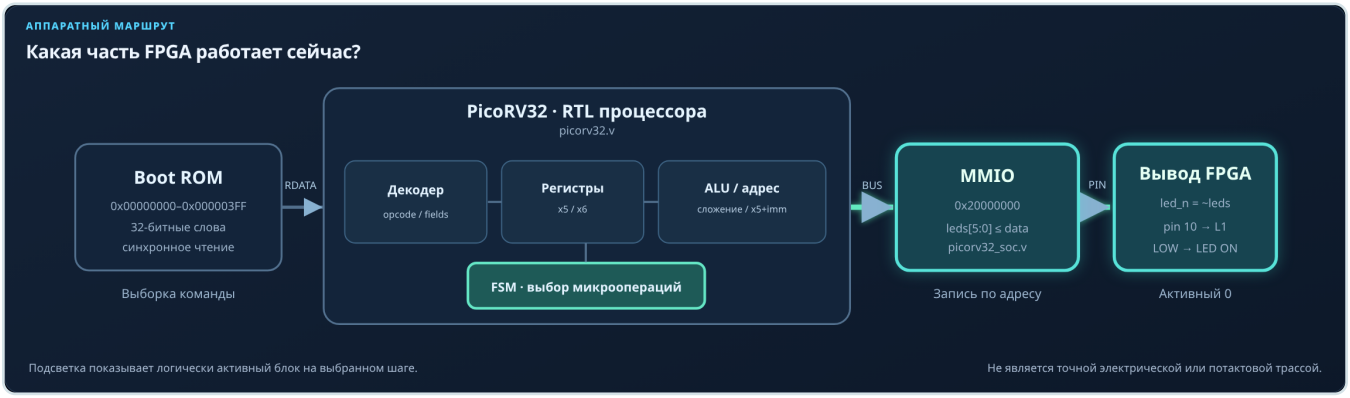
От машинного слова до светодиода L1

Иллюстрированное пошаговое руководство к интерактивной HTML-демонстрации PicoRV32 / RV32I на Tang Nano 9K

19 смысловых шагов | 4 инструкции RV32I | ROM → CPU → MMIO → LED



Функциональная схема из работающей HTML-демонстрации: активен путь от ядра к регистру LED и физическому выводу.

Малачевский А. А. / A. A. Malachevsky

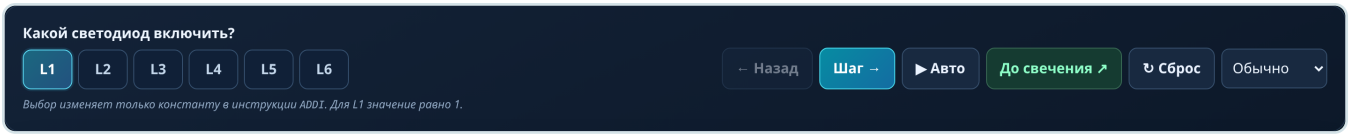
Независимый исследователь · ORCID 0009-0008-6009-3196
Проект: github.com/AIDevelopersMonster/Sipeed-FPGA

Руководство к автономному файлу `PicoRV32_LED1_Interactive_Demo_RU.html` · версия документа 1.0

ОРИЕНТАЦИЯ

Как пользоваться демонстрацией

Откройте **PicoRV32_LED1_Interactive_Demo_RU.html** в современном браузере двойным щелчком. Интернет, локальный сервер, компилятор и плата для работы HTML не требуются. Выберите **L1**, нажмите **«↻ Сброс»** и переходите дальше кнопкой **«Шаг →»**.



Настоящая панель управления: выбор L1–L6, «Назад», «Шаг», «Авто», «До свечения», «Сброс» и скорость.

Кнопки

- **Шаг →** — следующий смысловой этап.
- **← Назад** — предыдущий этап.
- **▶ Авто** — последовательное воспроизведение; выбор скорости справа.
- **До свечения ↗** — сразу перейти к подтверждённой записи SW (шаг 14).
- **↻ Сброс** — исходное состояние.
- **L1–L6** — выбрать светодиод; модель пересоздаёт машинное слово ADDI.

Пять панелей

- **Программа:** C и 4 строки ассемблера.
- **Машинное представление:** 32 бита, opcode, immediate.
- **CPU:** PC, x5, x6 и принятое слово.
- **Memory interface:** valid/ready, адрес, данные, strobes.
- **Светодиоды:** регистр leds и физические выходы.

> Главное условие чтения документа: **один шаг HTML не обязан быть одним тактом PicoRV32.** Это учебная последовательность смысловых действий, а не потактовая RTL-симуляция.

КОД И ПАМЯТЬ

Четыре слова в ROM — программа демонстрации

На языке C смысл операции выражает запись `*(volatile uint32_t*)0x20000000 = 1;`. HTML не утверждает, что конкретный C-компилятор выдаст именно четыре инструкции; ниже приведена специально подготовленная учебная программа RV32I.

```
_start:
    lui  t0, 0x20000
    addi t1, zero, 1
    sw   t1, 0(t0)
stop:
    jal  zero, stop
```

PC / адрес	Машинное слово	Мнемоника	Байты little-endian
0x00000000	**0x200002B7**	LUI x5,0x20000	B7 02 00 20
0x00000004	0x00100313	ADDI x6,x0,1	13 03 10 00
0x00000008	0x0062A023	SW x6,0(x5)	23 A0 62 00
0x0000000C	0x0000006F	JAL x0,0	6F 00 00 00

Откуда взялся 0x200002B7?

У формата U биты 31–12 содержат `0x20000`, биты 11–7 — `rd=5`, а биты 6–0 — `opcode=0110111` (LUI). Декодер превращает это в запись `0x20000000` в регистр x5.

А где сам светодиод?

`0x20000000` — адрес регистра LED, **не адрес команды**. Последующая SW запишет значение 1 в этот адрес, после чего `led_n[0]=0` на выводе FPGA 10.

> Инициализация CPU-регистров: после RESET содержимое x5 и x6 **не задано** при используемой конфигурации. Их определённые значения устанавливают инструкции LUI и ADDI. В отличие от них регистр leds в нашем SoC при reset обнуляется.

АППАРАТНЫЙ МАРШРУТ

Где проходят данные и кто чем управляет



Схема из HTML. Светящиеся блоки показывают логический маршрут выбранной микрооперации, но не точную электрическую трассу.

Блок	Назначение в примере L1
ROM	Выдаёт машинные слова по адресам 0x0, 0x4, 0x8 и 0xC.
Декодер PicoRV32	Определяет LUI, ADDI, SW, JAL по opcode, funct3 и полям immediate.
cpuregs / регистровый файл	x5 хранит адрес 0x20000000, x6 — значение 1.
ALU / FSM	Выполняют арифметику, формируют адрес SW и порядок микроопераций.
Memory interface	Передаёт адрес, данные и строб записи; mem_ready подтверждает приём.
MMIO / leds	Адресный декодер выбирает регистр leds, записываются шесть младших бит данных.
FPGA pin 10 / L1	Формируется led_n[0]=~leds[0]. LOW включает L1.

> Обратите внимание: здесь не используется готовая «инструкция включить LED». У RISC-V есть обычная SW, а назначение адреса 0x20000000 определяется **нашим Verilog SoC**, а не ISA процессора.

Запрос инструкции с PC=0x00000000

Действие студента. Нажмите кнопку «Шаг →» в первый раз. На полосе прогресса должно появиться «Шаг 1 из 19».

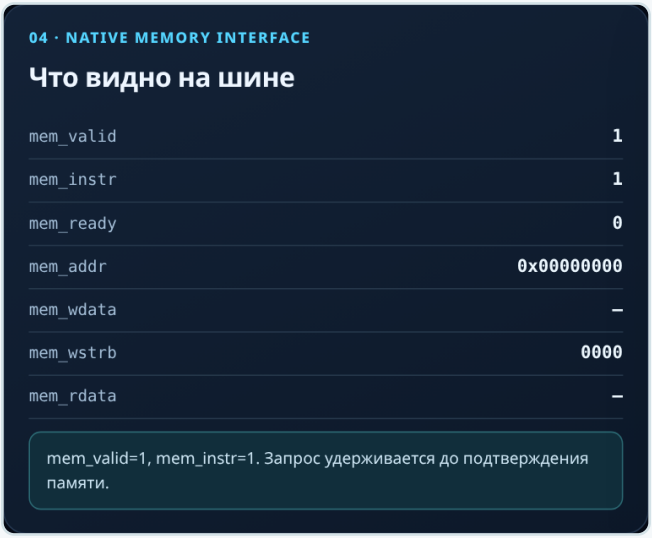
Что показывает HTML. Процессор выставляет адрес 0x00000000 и признак mem_instr=1. Это чтение команды из ROM, а не запись светодиода.

Что это означает в реальной цифровой системе

В момент запроса PC содержит ноль. PicoRV32 выставляет 32-битный адрес на native memory interface и поднимает mem_valid. mem_instr=1 уточняет: запрашивается инструкция. Это обращение к загрузочной ROM, а не к регистру LED. Для чтения mem_wstrb=0000.

Контрольные значения на экране

PC	0x00000000
mem_valid	1
mem_instr	1
mem_ready	0
mem_addr	0x00000000
mem_rdata	—



Панель интерфейса памяти в данном шаге

Главное для понимания: Адрес инструкции (0x00000000) не равен адресу LED-регистра (0x20000000).

Проверьте себя: Какой именно адрес сейчас выдаёт CPU: ROM или MMIO?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

ROM возвращает 0x200002B7

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 2 из 19».

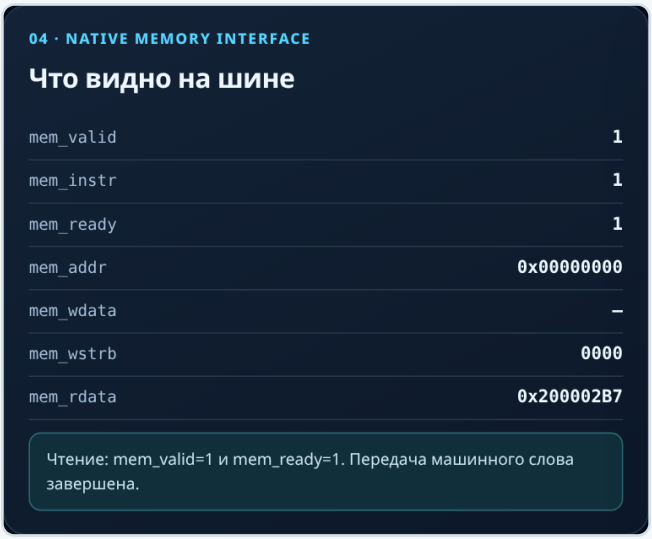
Что показывает HTML. Синхронная ROM выбирает ROM[0]. При активных mem_valid и mem_ready слово 0x200002B7 принимается процессором как инструкция.

Что это означает в реальной цифровой системе

Синхронная ROM выбирает слово с индексом mem_addr[9:2]=0 и возвращает 0x200002B7. Появление mem_ready=1 означает, что слово принято. Здесь меняется полученная инструкция, но не содержимое x5 и не уровень контакта FPGA.

Контрольные значения на экране

PC	0x00000000
mem_valid	1
mem_instr	1
mem_ready	1
mem_addr	0x00000000
mem_rdata	0x200002B7



Панель интерфейса памяти в данном шаге

Главное для понимания: Число в ROM — код инструкции, а не готовый адрес периферии.

Проверьте себя: Что означают слова mem_valid=1 и mem_ready=1 одновременно?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Декодер распознаёт LUI

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 3 из 19».

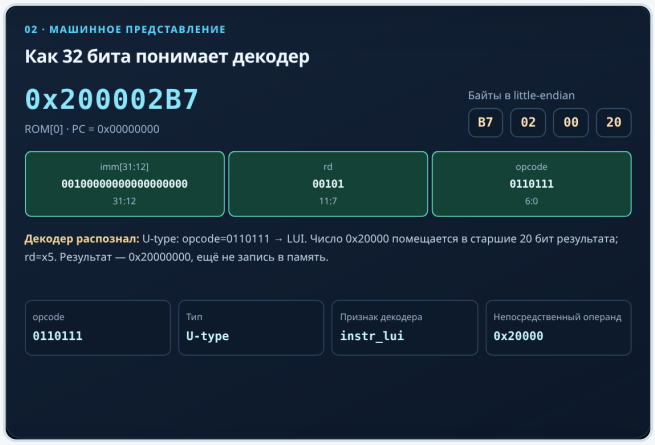
Что показывает HTML. По opcode и дополнительным полям логика определяет семейство U-type. instr_lui становится управляющим признаком выбранной операции.

Что это означает в реальной цифровой системе

Декодер отделяет младшие 7 бит: opcode=0110111 (LUI). Из бит 11:7 извлекается rd=00101, то есть x5. Биты 31:12 содержат 0x20000. В отличие от STORE, формат U не использует rs1 или rs2. Управление выбирает путь формирования верхней константы.

Контрольные значения на экране

Слово ROM	0x200002B7
opcode	0110111
Формат	U-type
Признак	instr_lui
imm	0x20000
PC	0x00000000



Панель декодирования текущего машинного слова

Главное для понимания: В LUI поле imm занимает биты 31:12, а rd — биты 11:7.

Проверьте себя: Какие биты 0x200002B7 определяют регистр x5?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

LUI: создаём адрес 0x20000000

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 4 из 19».

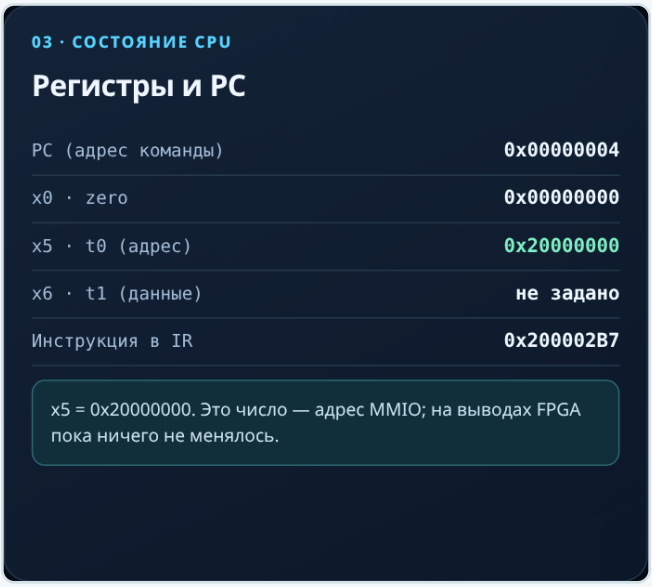
Что показывает HTML. Поле imm[31:12]=0x20000 сдвигается влево на 12. Значение записывается в x5 (t0). Светодиод всё ещё выключен.

Что это означает в реальной цифровой системе

LUI передаёт константу 0x20000 в старшие двадцать разрядов результата и обнуляет младшие 12: 0x20000 << 12 = 0x20000000. По разрешению записи в регистровый файл значение оказывается в x5 (t0), а PC смещается к следующему слову ROM — 0x00000004.

Контрольные значения на экране

PC	0x00000004
x5 (t0)	0x20000000
x6 (t1)	не задано
leds[5:0]	000000
led_n[5:0]	111111



Регистры и программный счётчик

Главное для понимания: В x5 записывается адрес; это ещё не обращение к периферии.

Проверьте себя: Почему значение 0x20000000 ещё не зажгло L1?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Запрос инструкции с PC=0x00000004

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 5 из 19».

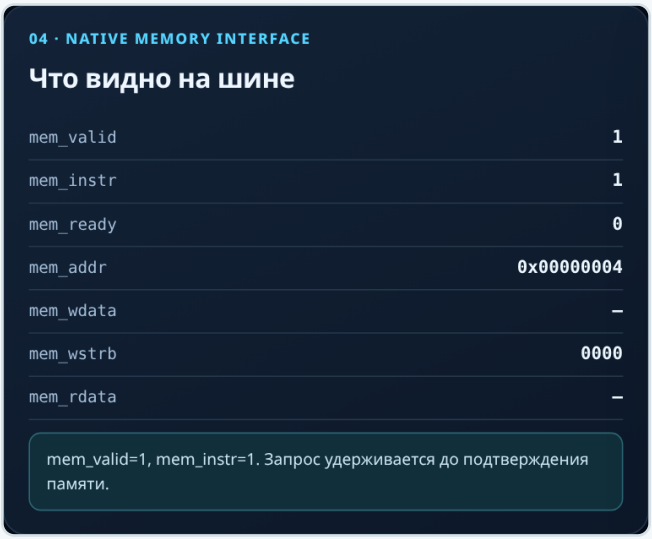
Что показывает HTML. Процессор выставляет адрес 0x00000004 и признак mem_instr=1. Это чтение команды из ROM, а не запись светодиода.

Что это означает в реальной цифровой системе

Теперь запрос направлен к ROM[1] по адресу PC=0x00000004. Адрес MMIO пока только записан в x5. Процессор не «обращается к светодиоду» в инструкции ADDI: он снова читает код из ROM через то же физическое соединение памяти.

Контрольные значения на экране

PC	0x00000004
mem_valid	1
mem_instr	1
mem_ready	0
mem_addr	0x00000004
mem_rdata	—



Панель интерфейса памяти в данном шаге

Главное для понимания: Каждый новый машинный код выбирается из ROM по текущему PC.

Проверьте себя: Почему CPU снова обращается к ROM, хотя адрес x5 уже готов?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

ROM возвращает 0x00100313

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 6 из 19».

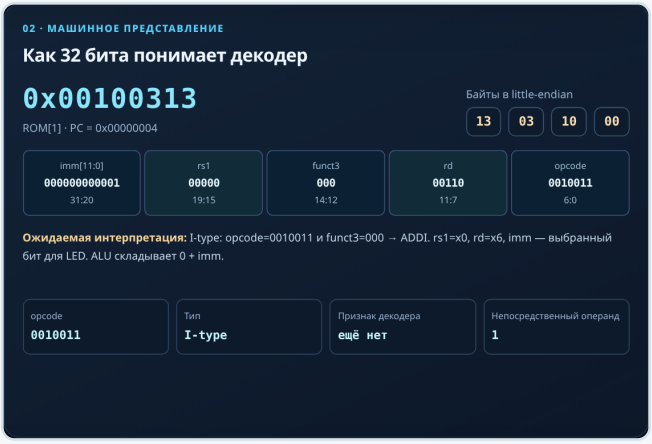
Что показывает HTML. Синхронная ROM выбирает ROM[1]. При активных mem_valid и mem_ready слово 0x00100313 принимается процессором как инструкция.

Что это означает в реальной цифровой системе

ROM[1] отдаёт 32-битное слово 0x00100313, которое соответствует ADDI. Байты этого слова в little-endian: 13 03 10 00. Пара valid/ready подтверждает завершение выборки команды; x6 пока не изменён.

Контрольные значения на экране

PC	0x00000004
mem_valid	1
mem_instr	1
mem_ready	1
mem_addr	0x00000004
mem_rdata	0x00100313



Панель интерфейса памяти в данном шаге

Главное для понимания: Для little-endian байты 0x00100313: 13 03 10 00.

Проверьте себя: Какому ассемблерному мнемонику соответствует 0x00100313?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Декодер распознаёт ADDI

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 7 из 19».

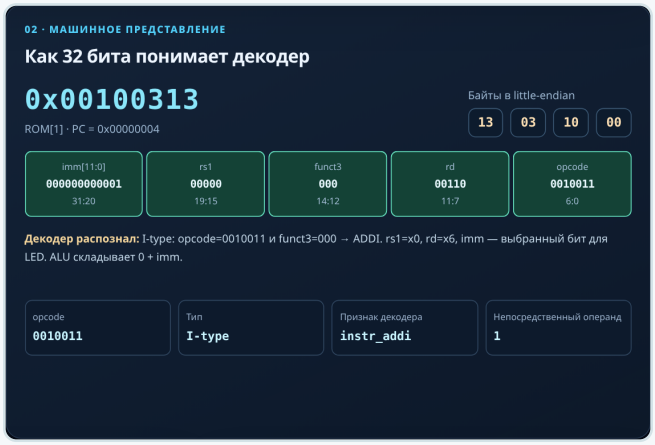
Что показывает HTML. По opcode и дополнительным полям логика определяет семейство I-type. instr_addi становится управляющим признаком выбранной операции.

Что это означает в реальной цифровой системе

Поле opcode=0010011 выбирает арифметику с immediate, funct3=000 — ADDI. Поля rs1=00000 и rd=00110 означают x0 и x6. Старшие 12 бит задают immediate=1. Значение x0 аппаратно всегда нулевое; внешний адрес памяти не используется.

Контрольные значения на экране

Слово ROM	0x00100313
opcode	0010011
Формат	I-type
Признак	instr_addi
imm	1
PC	0x00000004



Панель декодирования текущего машинного слова

Главное для понимания: В ADDI x0 неизменно ноль: ALU вычисляет 0+1.

Проверьте себя: Чем отличаются поля rs1 и rd в ADDI?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

ADDI: в x6 записываем 1

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 8 из 19».

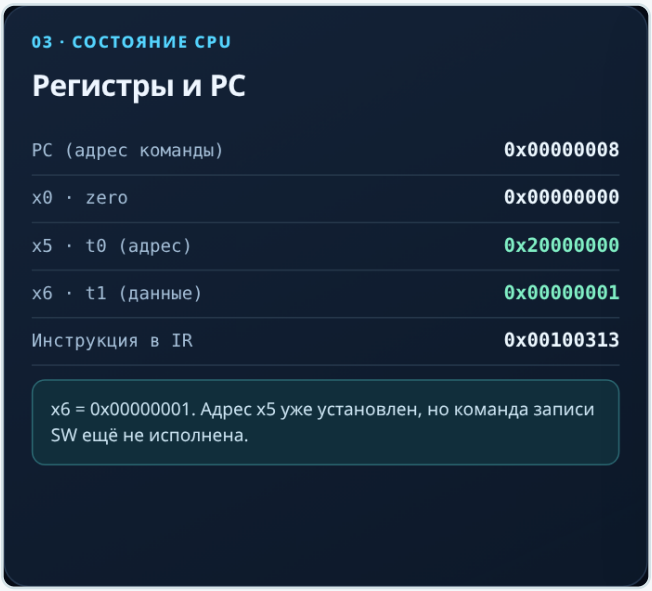
Что показывает HTML. ALU получает x0=0 и immediate=1, складывает их и сохраняет 0x00000001 в x6 (t1). Это будущие данные для LED-регистра.

Что это означает в реальной цифровой системе

ALU получает два операнда: 0 из x0 и immediate 1. Сумматор даёт 0x00000001, результат записывается в x6 (t1). Теперь подготовлены и адрес (x5), и данные (x6). Светодиоды выключены, потому что до этого момента транзакции STORE не было.

Контрольные значения на экране

PC	0x00000008
x5 (t0)	0x20000000
x6 (t1)	0x00000001
leds[5:0]	000000
led_n[5:0]	111111



Регистры и программный счётчик

Главное для понимания: В x6 теперь данные записи; LED всё ещё выключен.

Проверьте себя: В каком именно внутреннем регистре теперь лежит число 1?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Запрос инструкции с PC=0x00000008

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 9 из 19».

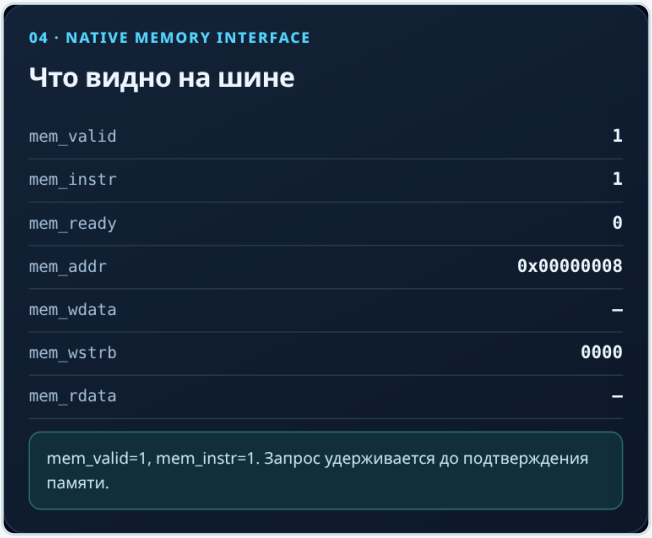
Что показывает HTML. Процессор выставляет адрес 0x00000008 и признак mem_instr=1. Это чтение команды из ROM, а не запись светодиода.

Что это означает в реальной цифровой системе

Третья команда находится в ROM[2] по PC=0x00000008. CPU начинает очередной запрос инструкции. Важно не смешивать адрес самой команды 0x00000008 и будущий адрес данных 0x20000000 — они принадлежат разным действиям одного процессора.

Контрольные значения на экране

PC	0x00000008
mem_valid	1
mem_instr	1
mem_ready	0
mem_addr	0x00000008
mem_rdata	—



Панель интерфейса памяти в данном шаге

Главное для понимания: Значение mem_addr=0x00000008 на данном шаге — адрес инструкции.

Проверьте себя: Почему 0x00000008 и 0x20000000 нельзя путать?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

ROM возвращает 0x0062A023

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 10 из 19».

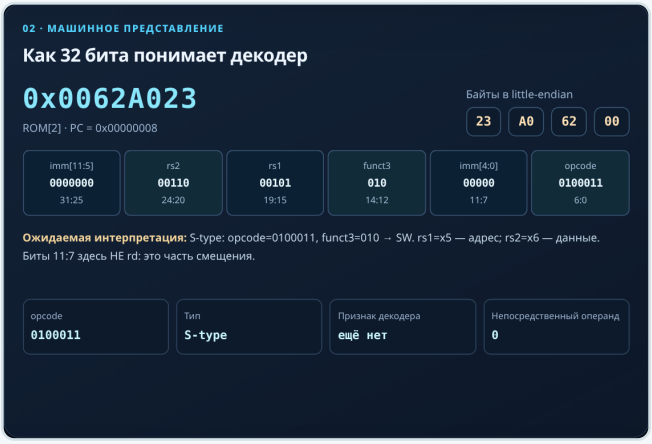
Что показывает HTML. Синхронная ROM выбирает ROM[2]. При активных mem_valid и mem_ready слово 0x0062A023 принимается процессором как инструкция.

Что это означает в реальной цифровой системе

Из ROM[2] возвращается 0x0062A023, то есть SW t1,0(t0). В little-endian слово представлено байтами 23 A0 62 00. Пока это лишь 32-битное число инструкции, которое ещё должно быть распознано декодером.

Контрольные значения на экране

PC	0x00000008
mem_valid	1
mem_instr	1
mem_ready	1
mem_addr	0x00000008
mem_rdata	0x0062A023



Панель интерфейса памяти в данном шаге

Главное для понимания: Биты 0x0062A023 сами не включают LED, пока CPU их не исполнит.

Проверьте себя: Какой код в ROM у команды SW и что такое little-endian?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Декодер распознаёт SW

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 11 из 19».

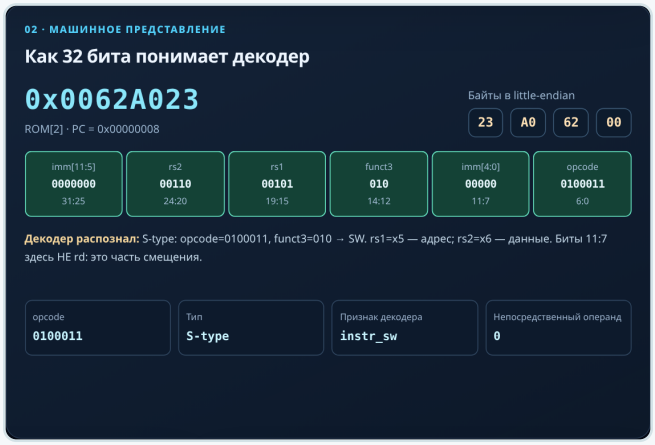
Что показывает HTML. По opcode и дополнительным полям логика определяет семейство S-type. instr_sw становится управляющим признаком выбранной операции.

Что это означает в реальной цифровой системе

Для формата S opcode=0100011 означает STORE, funct3=010 — SW (полное 32-битное слово). rs1=00101 выбирает x5, rs2=00110 выбирает x6. Поле imm собирается из бит 31:25 и 11:7; здесь оно равно нулю. У SW нет регистра назначения rd.

Контрольные значения на экране

Слово ROM	0x0062A023
opcode	0100011
Формат	S-type
Признак	instr_sw
imm	0
PC	0x00000008



Панель декодирования текущего машинного слова

Главное для понимания: У S-type биты 11:7 не являются rd — это часть immediate.

Проверьте себя: Почему у SW нет rd, несмотря на присутствие бит 11:7?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

SW: откуда брать адрес и данные?

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 12 из 19».

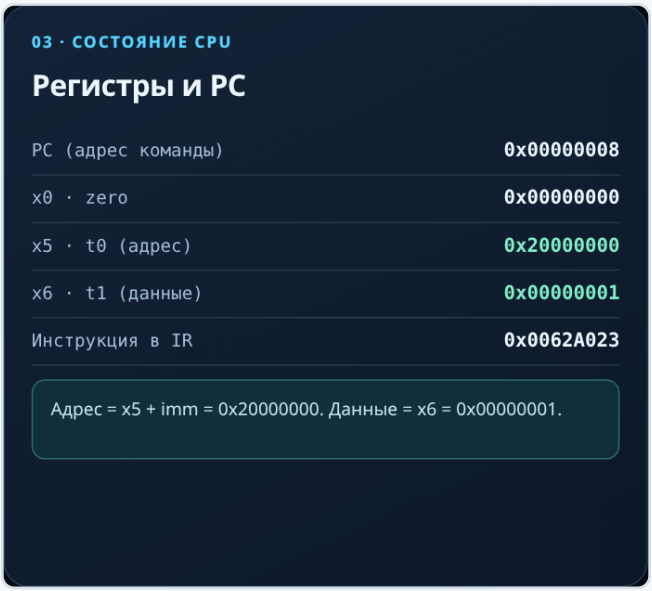
Что показывает HTML. Декодер извлёк rs1=5 (x5), rs2=6 (x6), imm=0. Регистровый файл выдаёт x5=0x20000000 и x6=0x00000001. Адресный тракт вычисляет x5+0.

Что это означает в реальной цифровой системе

Тракт адреса принимает x5=0x20000000 и вычисляет эффективный адрес x5+sign_extend(0). Данные для записи поступают из x6=1. Реальное ядро использует управление cpu_state_stmem; на учебном экране этот момент выделен отдельно от самой транзакции.

Контрольные значения на экране

PC	0x00000008
x5 (t0)	0x20000000
x6 (t1)	0x00000001
leds[5:0]	000000
led_n[5:0]	111111



Регистры и программный счётчик

Главное для понимания: Декодер выбирает SW, а адресный тракт формирует x5+0.

Проверьте себя: Откуда берутся адрес и данные при выполнении SW?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

SW выдаёт запись по адресу 0x20000000

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 13 из 19».

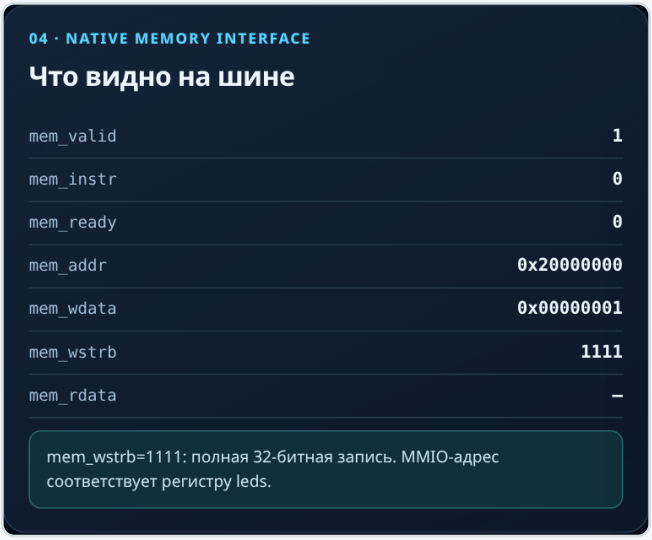
Что показывает HTML. PicoRV32 устанавливает mem_valid=1, mem_instr=0, mem_wstrb=1111, mem_addr=0x20000000 и mem_wdata=0x00000001. MMIO декодирует диапазон и адрес LED-регистра.

Что это означает в реальной цифровой системе

На внешнем интерфейсе выставлены mem_valid=1, mem_instr=0, mem_addr=0x20000000, mem_wdata=0x00000001 и mem_wstrb=1111. Старшие биты адреса активируют is_mmio, младший байт адреса выбирает регистр leds. Пока mem_ready=0, запрос не считается завершённым.

Контрольные значения на экране

mem_valid	1
mem_ready	0
mem_instr	0
mem_addr	0x20000000
mem_wdata	0x00000001
mem_wstrb	1111
leds[5:0]	000000



Параметры текущего состояния

Главное для понимания: При mem_ready=0 LED-регистр ещё не обновлён.

Проверьте себя: Что произойдёт, если mem_ready останется 0?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Загорелся L1: выход pin 10 стал LOW

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 14 из 19».

Что показывает HTML. На активном фронте в регистр leds[5:0] записывается 000001. Провод led_n[0]=~leds[0] переходит в 0. На запрос SW выдаётся mem_ready=1; запись принята.

Что это означает в реальной цифровой системе

На активном фронте логика SoC выполняет leds <= mem_wdata[5:0], поэтому leds=000001. Одновременно она выставляет зарегистрированный mem_ready. Комбинационное assign led_n=~leds формирует led_n=111110: бит led_n[0] равен нулю. Физический вывод FPGA 10 переходит в LOW — L1 загорается.

Контрольные значения на экране

mem_valid	1
mem_ready	**1**
mem_instr	0
mem_addr	**0x20000000**
mem_wdata	**0x00000001**
mem_wstrb	1111
leds[5:0]	**000001**



Физическое состояние LED и выводов FPGA

Главное для понимания: Новый leds=000001 инвертируется в физические выходы 111110.

Проверьте себя: Почему логическая единица leds[0] соответствует LOW на pin 10?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Запись завершена; L1 продолжает гореть

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 15 из 19».

Что показывает HTML. Процессор снимает запрос и переходит к PC=0x0000000C. LED-регистр сохраняет бит=1; подсветка не зависит от дальнейших команд CPU.

Что это означает в реальной цифровой системе

После подтверждения CPU завершает операцию SW: сигнал запроса снимается, PC готов к адресу 0x0000000C. Шесть триггеров led-регистра сохраняют 000001. Состояние выхода не зависит от того, что процессор теперь исполняет другую инструкцию.

Контрольные значения на экране

PC	0x0000000C
x5 (t0)	0x20000000
x6 (t1)	0x00000001
leds[5:0]	000001
led_n[5:0]	111110



Физическое состояние LED и выводов FPGA

Главное для понимания: Триггеры периферии удерживают значение независимо от дальнейшего PC.

Проверьте себя: Почему светодиод горит без повторения SW?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Запрос инструкции с PC=0x0000000C

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 16 из 19».

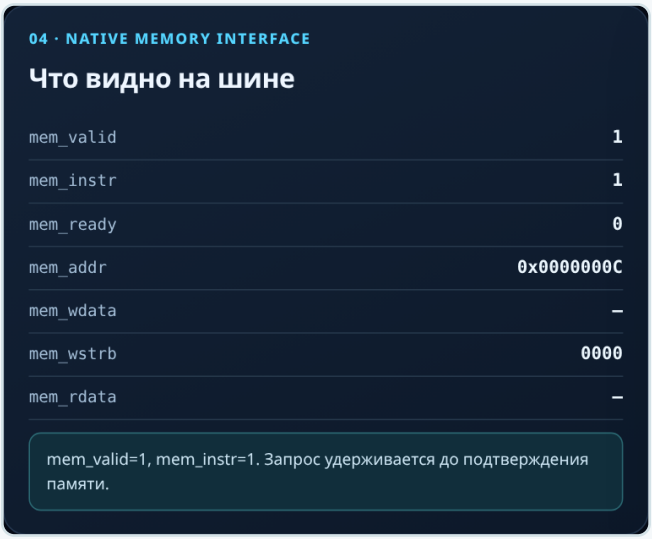
Что показывает HTML. Процессор выставляет адрес 0x0000000C и признак mem_instr=1. Это чтение команды из ROM, а не запись светодиода.

Что это означает в реальной цифровой системе

Процессор запрашивает ROM[3] по PC=0x0000000C. Состояние leds не меняется, потому что чтение инструкции не обращается к MMIO. Это наглядно показывает разделение хранения состояния периферии и исполнения кода процессором.

Контрольные значения на экране

PC	0x0000000C
mem_valid	1
mem_instr	1
mem_ready	0
mem_addr	0x0000000C
mem_rdata	—



Панель интерфейса памяти в данном шаге

Главное для понимания: Выборка JAL не меняет состояние периферии.

Проверьте себя: Что в этой выборке отличается от чтения данных по MMIO?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

ROM возвращает 0x0000006F

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 17 из 19».

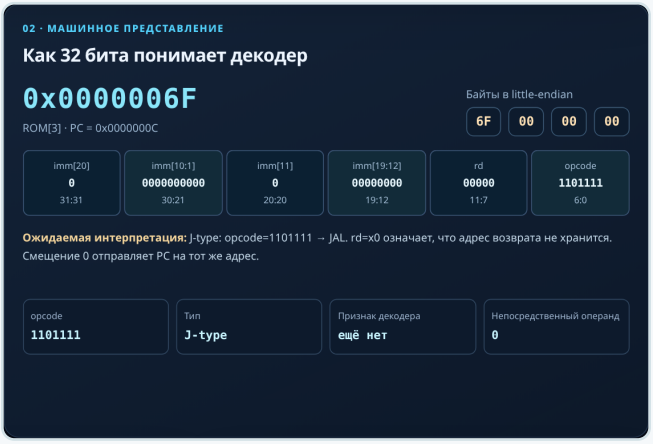
Что показывает HTML. Синхронная ROM выбирает ROM[3]. При активных mem_valid и mem_ready слово 0x0000006F принимается процессором как инструкция.

Что это означает в реальной цифровой системе

ROM[3] возвращает 0x0000006F. Это инструкция JAL с нулевым смещением. Пара mem_valid/mem_ready обслуживает выборку так же, как для предыдущих инструкций; сохранённое значение leds не затрагивается.

Контрольные значения на экране

PC	0x0000000C
mem_valid	1
mem_instr	1
mem_ready	1
mem_addr	0x0000000C
mem_rdata	0x0000006F



Панель интерфейса памяти в данном шаге

Главное для понимания: JAL закодирована 0x0000006F; PC текущей команды равен 0x0000000C.

Проверьте себя: Какой машинный код имеет JAL с нулевым смещением?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Декодер распознаёт JAL

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 18 из 19».

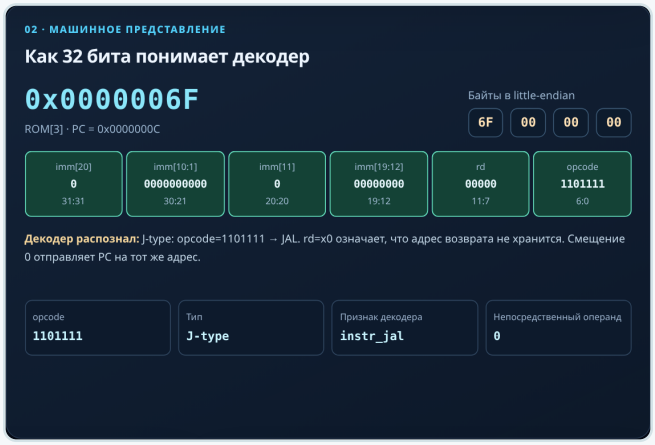
Что показывает HTML. По opcode и дополнительным полям логика определяет семейство J-type. instr_jal становится управляющим признаком выбранной операции.

Что это означает в реальной цифровой системе

Поле opcode=1101111 обозначает JAL, поле rd=00000 — x0. Все биты immediate образуют смещение 0. Поскольку адрес возврата записывался бы в x0, запись отбрасывается. Для ветвления JAL не читает и не записывает MMIO-регистры.

Контрольные значения на экране

Слово ROM	0x0000006F
opcode	1101111
Формат	J-type
Признак	instr_jal
imm	0
PC	0x0000000C



Панель декодирования текущего машинного слова

Главное для понимания: rd=x0 у JAL означает отказ от сохранения адреса возврата.

Проверьте себя: Куда попал бы адрес возврата, если бы rd не был x0?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

Процессор повторяет JAL, а L1 остаётся включённым

Действие студента. Нажмите кнопку «Шаг →» ещё один раз. На полосе прогресса должно появиться «Шаг 19 из 19».

Что показывает HTML. Инструкция JAL x0,0 задаёт смещение 0: PC остаётся 0x0000000C. Ядро продолжает выбирать и исполнять эту команду снова и снова; светодиод остаётся включённым, потому что никто не меняет leds[0].

Что это означает в реальной цифровой системе

JAL вычисляет новый PC как текущий PC+0, то есть 0x0000000C. Процессор будет снова выбирать и исполнять эту же инструкцию, а HTML останавливается на первом обходе. Светодиод остаётся включённым, пока программа или сброс не изменят регистр leds.

Контрольные значения на экране

PC	0x0000000C
x5 (t0)	0x20000000
x6 (t1)	0x00000001
leds[5:0]	000001
led_n[5:0]	111110



Физическое состояние LED и выводов FPGA

Главное для понимания: Бесконечный цикл CPU не является новым циклом записи в LED.

Проверьте себя: Почему бесконечное выполнение JAL не выключает L1?

Все сведения здесь описывают смысловой шаг HTML. Это не один гарантированный такт конкретного RTL и не измерение временных задержек FPGA.

САМОСТОЯТЕЛЬНАЯ ПРОВЕРКА

Изменяем L1 на L2–L6

Вверху HTML выберите другой светодиод. Меняется константа во второй инструкции ADDI, но базовый адрес MMIO и машинное слово SW остаются прежними. Для L4, например, используется число 8: x6=8, а физический вывод 14 переходит в LOW.

LED	Pin	ADDI код	x6	leds[5:0]	led_n[5:0]
1	`pin 10`	00100313	0x00000001	000001	111110
2	`pin 11`	00200313	0x00000002	000010	111101
3	`pin 13`	00400313	0x00000004	000100	111011
4	`pin 14`	00800313	0x00000008	001000	110111
5	`pin 15`	01000313	0x00000010	010000	101111
6	`pin 16`	02000313	0x00000020	100000	011111

Что должен сделать студент

1. Нажать «↻ Сброс», выбрать L4. Убедиться, что значение x6 ещё не задано.
2. Перейти к шагу 8. Проверить x6=0x00000008 и машинное слово 0x00800313.
3. Перейти к шагу 13. Убедиться, что mem_wdata=0x00000008, но mem_ready ещё ноль.
4. Перейти к шагу 14. Проверить leds=001000, led_n=110111, pin 14 LOW.
5. Объяснить, почему регистры x5 и x6 не содержат номер физического вывода 14 как адрес.

> Этот опыт — изменение модели в браузере, а не запись на настоящую плату. Для аппаратного эксперимента понадобятся новая ROM, пересборка .fs и независимый тест RTL.

УРОВЕНЬ РАЗРАБОТКИ

Соответствие HTML настоящему Verilog

1. Загрузочная ROM

```
boot_rom u_rom (.clk(clk), .word_addr(mem_addr[9:2]), .rdata(rom_data));
// ROM[0] = 32'h200002B7, ROM[1] = 32'h00100313 ...
```

Наше учебное четырёхсловное содержание отличается от действующей лабораторной прошивки из 33 слов. Для сопоставления архитектуры используется тот же способ ROM-выборки и MMIO.

2. Признаки декодера и путь STORE

```
decoded_rs1 <= mem_rdata_latched[19:15];
decoded_rs2 <= mem_rdata_latched[24:20];
// Для SW opcode=7'b0100011, funct3=3'b010.
```

3. Подтверждение MMIO и выход на LED

```
wire is_mmio = (mem_addr[31:8] == 24'h200000);
// По адресу 0x20000000:
if (|mem_wstrb && mem_wstrb[0]) leds <= mem_wdata[5:0];
mem_ready <= 1'b1;
assign led_n = ~leds;
```

Три важных разграничения

1. **Модель ≠ RTL-симулятор:** 19 этапов не эквивалентны 19 тактам, HTML не запускает исходный Verilog.
2. **IR на экране — учебное представление принятой инструкции:** этот термин не доказывает наличия отдельного одноимённого регистра в RTL ядра.
3. **Подтверждённая аппаратура ≠ новый пример:** 19 октября 2026 года на Tang Nano 9K прошёл тест исходной программы из 33 слов (`PICO` по UART и анимация LED). Отдельный 4-командный пример, показанный здесь, аппаратно не проверялся.

КОНТРОЛЬ УСВОЕНИЯ

Вопросы к студенту и источники

Вопросы по пониманию

1. Где хранится инструкция 0x200002B7?
2. Почему x5 после LUI равно 0x20000000?
3. В чём разница между mem_addr на шаге 9 и шаге 13?
4. Как распознать SW по полям 0x0062A023?
5. Когда запрос SW считается завершённым?
6. Почему leds[0]=1 приводит к LOW?
7. Почему после JAL светодиод продолжает светиться?

Контрольный отчёт

1. Указать выбранный LED и физический pin.
2. Привести 4 слова ROM с адресами.
3. Объяснить содержимое x5 и x6 после шага 8.
4. Записать все сигналы на шагах 13 и 14.
5. Показать инверсию leds → led_n.
6. Сформулировать пределы HTML-модели.

Ссылки на исходники

Проект: https://github.com/AIDevelopersMonster/Sipeed-FPGA
Исходное ядро: https://github.com/YosysHQ/picorv32
Файлы SoC: 01-PicoRV32/rtl/picorv32_soc.v, boot_rom.v, constraints/tang_nano_9k.cst.

Атрибуция: автор HTML-демонстрации и настоящего руководства — Малачевский А. А. / A. A. Malachevsky (ORCID: 0009-0008-6009-3196). Исходный PicoRV32 — © Claire Xenia Wolf, лицензия ISC; авторство ядра не присваивается автору данной учебной интеграции.